Data-driven Control
Integration week

I. Project Description

The end of your project is approaching, and the ground drone you've developed is now complete. You are eagerly awaiting Friday's demonstration, where you'll present your work to the client.

During a trade show, your client mentioned a key limitation of ground drones: *"Ground drones have limited field of view for obstacle detection. It would be interesting to add an aerial perspective to improve this capability."*

Back from the trade show, you discussed the idea with your colleagues and decided to implement a solution immediately. A quadrotor drone will be added to a platform at the back of the ground vehicle. It will be able to take off, gather data from above, and land again. A schematic of this idea is shown in Figure 1.

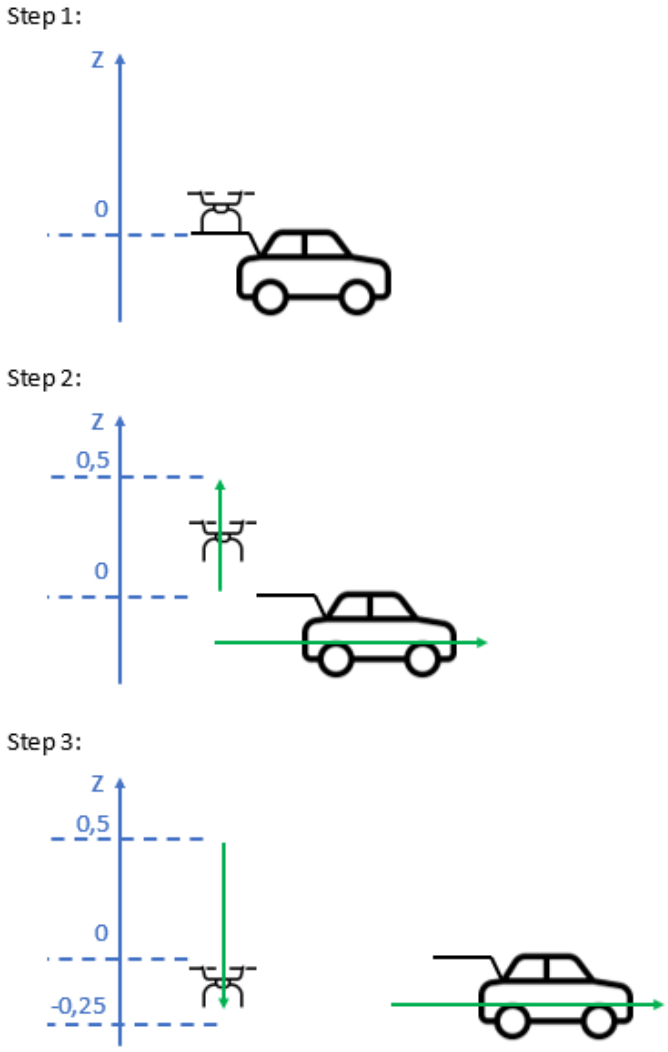


Fig. 1. Project Overview

You contact your supplier, who can provide a quadrotor drone today, but unfortunately, the documentation is missing. Thankfully, having taken the "Data-Driven Control" course, you're confident you can develop the required control law using data alone.

With the drone, you've also recovered a set of recorded measurements. You recall that they were collected using a somewhat unreliable sensor, so you'll likely need to verify their quality first.

II. Expected Work

You are expected to complete the following steps:

1) Define a model for the drone:
   - Justify the sampling period ($T_s = 0.05$s)
   - Choose an appropriate model[1].
2) Validate the dataset (removal of outliers, normalization, etc.).
3) Identify the model.
4) Develop a control law. The drone should follow the steps illustrated in Figure 1.

Steps 1 to 3 are common to the whole class. For step 4, the class is split into two groups: half of the groups follow Track A (MPC), the other half Track B (reinforcement learning), described in Section III.

III. Control Law: Two Tracks

The set of groups will be split in two subsets, each studying a different advanced control technique: MPC and RL.

A. Track A: Model Predictive Control

Model Predictive Control (MPC) is expected, using the identified model as the prediction model. An LQ controller on a linear model can also be used for comparison.

B. Track B: Reinforcement Learning

A Reinforcement Learning (RL) controller is expected, trained in Python using the library stable-baselines3. It is trained on the model you identified in steps 1 to 3, whatever its form, as long as it can be run from Python (if you identified it in MATLAB: copy the coefficients of a linear model, export the network to ONNX, or identify it again in Python). You will have to set up your own environment, reward structure and choose the right RL algorithm.

If the results are not up to expectation, we expect critical thinking from you to explain the discrepancies. Don't hesitate to compare the RL controller to any linear baseline (PID, LQ...).

As this task is a bit more exploratory, some files are provided to help you (see below).

IV. Advanced tasks

If you finish early, you can continue with the following advanced tasks (ordered by increasing difficulty):

- Make the drone rotate on itself at the end of step 2.
- Make the drone land at a different position (e.g., by moving along the x axis).
- Make the drone follow the ground vehicle.
- Make the drone land on the moving ground drone.

V. Provided Files

The following files are provided:

- *bdd_in_mat_05.csv* and *bdd_out_mat_05.csv* — input/output datasets to be used (you may add more data to improve the model).
- *UAV_model.slx* — the system model to be studied (it calls the protected *quadcopter_model.p*).
- *EINT_system_simulation.slx* — an example Simulink usage of the system.
- *random_init.m* — initialization of the simulation: sampling period, noise gain `g_noise` and random seeds.
- Track B only: *rl_interface/* — the tools to export the policy you trained in Python to MATLAB, and to fly it on *UAV_model.slx* (instructions in *track_b.ipynb*).

[1] A neural network model is expected, but it may be relevant to compare it with a linear model.